

Московский авиационный институт
(национальный исследовательский университет)

Факультет Компьютерные науки и прикладная математика

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Дискретный анализ»

Студент: Савельев Артем
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 11.04.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2

Условие задачи:

Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64} - 1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

+ word 34 — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.

- word — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.

word — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».

Вариант сортировки: AVL дерево.

1 Описание

Требуется реализовать программную библиотеку для словаря на основе ****AVL-дерева**** — самобалансирующегося бинарного дерева поиска. AVL-дерево гарантирует высоту $\log n$ и время выполнения операций вставки, удаления и поиска $O(\log n)$ за счёт автоматической балансировки после каждой операции (разница высот левого и правого поддеревьев не превышает 1).

Для ключей (регистронезависимые строки английских букв длиной ≤ 256) сравнение и хранение выполняются в нижнем регистре. Значением является 64-битное беззнаковое целое число.

Реализация содержит шаблонный класс `avl::AVL` с необходимыми поворотами (`left`, `right`, `left-right`, `right-left`) и вспомогательные функции для удаления.

```
1  class AVL {
2      private:
3          struct Node {
4              Key key;
5              Value value;
6              int height = 1;
7              Node* left = nullptr;
8              Node* right = nullptr;
9
10             Node(const Key& k, const Value& v) : key(k), value(v) {}
11 };
12
13 Node* root_ = nullptr;
14 size_t size_ = 0;
15
16 int get_height(Node* node) const { return node ? node->height : 0;
17     }
18
19 int balance_factor(Node* node) const {
20     return node ? get_height(node->right) - get_height(node->left)
21         : 0;
22 }
23
24 void fix_height(Node* node) {
25     if (!node) return;
26     int hl = get_height(node->left);
27     int hr = get_height(node->right);
28     node->height = (hl > hr ? hl : hr) + 1;
29 }
30
31 Node* rotate_right(Node* p) {
32     Node* q = p->left;
33     p->left = q->right;
34     q->right = p;
35     fix_height(p);
36     fix_height(q);
37     return q;
38 }
39
40 Node* rotate_left(Node* q) {
41     Node* p = q->right;
```

```

40     q->right = p->left;
41     p->left = q;
42     fix_height(q);
43     fix_height(p);
44     return p;
45 }
46
47 Node* balance(Node* p) {
48     if (!p) return nullptr;
49     fix_height(p);
50
51     if (balance_factor(p) == 2) {
52         if (balance_factor(p->right) < 0) p->right =
53             rotate_right(p->right);
54         return rotate_left(p);
55     }
56     if (balance_factor(p) == -2) {
57         if (balance_factor(p->left) > 0) p->left =
58             rotate_left(p->left);
59         return rotate_right(p);
60     }
61     return p;
62 }
63
64 Node* insert(Node* node, const Key& key, const Value& value,
65             bool& success) {
66     if (!node) {
67         success = true;
68         ++size_;
69         return new Node(key, value);
70     }
71     if (key < node->key)
72         node->left = insert(node->left, key, value, success);
73     else if (key > node->key)
74         node->right = insert(node->right, key, value, success);
75     else
76         success = false;
77     return balance(node);
78 }
79
80 Node* remove_min(Node* node, Node*& min_node) {
81     if (!node->left) {
82         min_node = node;
83         return node->right;
84     }
85     node->left = remove_min(node->left, min_node);
86     return balance(node);
87 }
88
89 Node* remove(Node* node, const Key& key, bool& success) {
90     if (!node) {

```

```

91         success = false;
92         return nullptr;
93     }
94
95     if (key < node->key)
96         node->left = remove(node->left, key, success);
97     else if (key > node->key)
98         node->right = remove(node->right, key, success);
99     else {
100         success = true;
101         --size_;
102
103         if (!node->left || !node->right) {
104             Node* temp = node->left ? node->left : node->right;
105             delete node;
106             return temp;
107
108         } else {
109             Node* min_node = nullptr;
110             node->right = remove_min(node->right, min_node);
111
112             min_node->left = node->left;
113             min_node->right = node->right;
114
115             delete node;
116             return balance(min_node);
117         }
118     }
119     return balance(node);
120 }
121
122 // ... etc.
123
124 void serialize(Node* node, std::ofstream& out) const {
125     if (!node) return;
126
127     uint16_t len = static_cast<uint16_t>(node->key.size());
128     out.write(reinterpret_cast<const char*>(&len), sizeof(len));
129     out.write(node->key.data(), len);
130     out.write(reinterpret_cast<const char*>(&node->value),
131              sizeof(node->value));
132
133     serialize(node->left, out);
134     serialize(node->right, out);
135 }
136
137 // ... etc.
138 }

```

2 Выводы

Выполнив вторую лабораторную работу по курсу «Дискретный анализ», я изучил принципы работы самобалансирующихся бинарных деревьев поиска и реализовал AVL-дерево для структуры данных словаря.

В процессе выполнения лабораторной работы я:

- Реализовал AVL-дерево с поворотами и балансировкой.
- Научился работать с сериализацией.
- Реализовал обработку входных команд (+, -, поиск) с корректной обработкой всех случаев.
- Написал Python-скрипты для генерации тестов и автоматической проверки.
- Провёл сравнительный анализ производительности с `std::map` (бенчмарк).

Результаты бенчмарков подтвердили, что собственная реализация AVL-дерева работает на уровне стандартного красно-чёрного дерева `std::map`.

3 Тест производительности

Тест производительности сравнивает время работы реализованного AVL-дерева (`avl::AVL`) с библиотечной структурой `std::map` (красно-чёрное дерево) из STL C++.

```
1  artems@MacBook-Air-artem-3 lab2 % ./benchmark
2  === BENCHMARK RESULTS ===
3  Total Operations : 177178
4  -----
5  std::map (RB) : 84 ms | Mismatches: 0
6  avl::AVL : 77 ms | Mismatches: 0
7  -----
8  SUCCESS: Both trees produced 100% correct results.
9
10 === BENCHMARK RESULTS ===
11 Total Operations : 371753
12 -----
13 std::map (RB) : 215 ms | Mismatches: 0
14 avl::AVL : 204 ms | Mismatches: 0
15 -----
16 SUCCESS: Both trees produced 100% correct results.
17
18 artems@MacBook-Air-artem-3 lab2 % ./benchmark tests/01
19 Loading data into memory... Done. Loaded 943583 operations.
20 Running std::map... Done.
21 Running avl::AVL... Done.
22 === BENCHMARK RESULTS ===
23 Total Operations : 943583
24 -----
25 std::map (RB) : 806 ms | Mismatches: 0
26 avl::AVL : 818 ms | Mismatches: 0
27 -----
28 SUCCESS: Both trees produced 100% correct results.
```

Реализованное AVL-дерево показывает производительность на уровне `std::map` даже на почти миллионе операций, что подтверждает эффективность реализации.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ*, 3-е издание. — Издательский дом «Вильямс», 2013. — 1328 с.
- [2] *C++ reference*.
URL: <https://en.cppreference.com/>